

(在本次實驗中, 由於時間問題, 來不及攝影相關照片, 所以後來另外用edsim51di下去做相關模擬截圖)

問題&心得:

- (1) 在lab01部分有出現一樣的格式error, 是程式一開始要初始化的code"ORG 0H"需要去注意他的排版對齊, 那再來基本上在lab01基本版以及bonus01沒有額外的問題。
- (2) 而在bonus02的部分比較有問題, 在一開始我們這組做的方向有誤(所以花的時間相對較多, 在本週沒有demo完), 有誤的地方是我們理解成, 將專門存放紀錄溢位結果的register中的值(次數)做為判斷條件, 如果值不等於零的話先亮一顆, 再將值做一次比較, 不等於1的話亮第二顆, 以上是我們原先理解的錯誤觀念; 正確結果應該要是判斷值有幾個(多少), 就亮幾顆led燈, 不是最多亮兩顆, 最多會亮8顆(如果溢位次數是8的話)。
- (3) 本次實驗中我學會了如何去運用指令做出相關的應用, 此次應用中有做出乘加運算以及溢位檢察, (bonus02)只是在操作微算機開發板時有遇到, 一是找不到COM4連接埠(已解決), 二是SIMLAB8051的模擬有亮全部八顆燈(bonus02), 但在實際連接上微算機開發版後會有兩顆燈亮不起來, 我和組員打算在demo前再次確認這個部分的問題。

本次實驗內容結果:

1. lab01基本:

(1) 作法:

- 有使用到的指令: MOV(資料搬移)、MUL AB(乘法, 結果放在B:A)、ADD、INC(加一)、DJNZ(遞減後若不是0就跳躍)、SJMP(無條件短跳躍)。
- 程式寫法:
 - a. 程式起始: 先用ORG 0000H設定好程式的起始位址。
 - b. 暫存器準備: R0、R1再分別指向A和B的起始位址(0x20、0x28), 作為指標, R2放入8當計數器。
 - c. 初始化陣列: 先將register A設定為1, 再用迴圈並透過MOV @R0, A、MOV @R1, A依序寫入, 將兩個陣列(A[], B[])初始化為1, 並再用INC移到下一格, 直到總共寫入8筆資料。
 - d. 指標重設: 重新設定R0、R1指標回到A、B的起點, R2再設8, 並且清除R3為0作為累加器使用。
 - e. 進入主要計算迴圈:

每次都先取出A[i]、B[i]存到A、B, 再執行MUL AB做8-bit*8-bit, 低位會在A, 高位會在B, 取出低位的結果A, 將A與目前總和R3相加去更新和存回到R3, 讓R0、R1移動到下一筆資料(各自加一), 再透過DJNZ R2, L0去控制迴圈直到總共跑滿了8次。
 - f. 結尾: 所有乘積相加完後, 程式跳到SJMP WAIT進入無窮迴圈, 讓程式停在結果狀態。

(2) 程式&結果照片:

- 結果: 程式成功將A陣列的20H~27H和B陣列的28H~2FH各8格位址初始化為1, 進入主要迴圈後, 程式將其依序計算A[i]*B[i]並累加存進R3, 又由於每次乘積皆為1、總共8筆資料, 因此最終結果會是和8(08H)存回R3, 在Data memory模擬中可見此結果。

ORG 0H			
000001	MOV R0,#20H	000121	MOV R2,#08H
000021	MOV R1,#28H	000141	MOV R3,#00H
000041	MOV R2,#08H	000161 L0:	MOV A,@R0
000061	MOV A,#01H	000171	MOV B,@R1
000081 I0:	MOV @R0,A	000191	MUL AB
000091	MOV @R1,A	0001A1	ADD A,R3
0000A1	INC R0	0001B1	MOV R3,A
0000B1	INC R1	0001C1	INC R0
0000C1	DJNZ R2,I0	0001D1	INC R1
0000E1	MOV R0,#20H	0001E1	DJNZ R2,L0
000101	MOV R1,#28H	000201 WAIT:	SJMP WAIT
000121	MOV R2,#08H	END	

Data Memory		addr		0x00		0x00		value									
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	28	30	00	08	00	00	00	00	00	00	00	00	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01
30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

2. bonus_1:

(1) 作法:

- 程式寫法和主要流程基本上和lab01基本版是一樣的(陣列初始化、乘加迴圈、最後停在無窮迴圈中)。
- 程式寫法(和lab01基本版不同的地方):
 - a. 新增變數N: 在程式一開始用MOV 30H, #05H將N=5存入內部RAM的30H位址, (N可以是2~7)。
 - b. 修改迴圈次數設定: 將原本寫死的MOV R2, #08H改成MOV R2, 30H, 讓程式從RAM去讀取N的值作為迴圈次數, 且初始化陣列以及乘加迴圈都改一致使用MOV R2, 30H, 確保運算次數會和變數N使一致的。

(2) 程式&結果照片:

- 結果:程式將A陣列的20H~24H和B陣列的28H~2CH初始化為1, N=5則是存放在RAM的30H處, 主迴圈執行5次乘加後, 累加的結果和為5(R3=05H)。

ORG 00H

00001 MOV 30H,#05H

00031 MOV R0,#20H

00051 MOV R1,#28H

00071 MOV R2,30H

00091 MOV A,#01H

000B10: MOV @R0,A

000C1 MOV @R1,A

000D1 INC R0

000E1 INC R1

000F1 DJNZ R2,L0

00111 MOV R0,#20H

00131 MOV R1,#28H

00151 MOV R2,30H

00171 MOV R3,#00H

00191 L0: MOV A,@R0

001A1 MOV B,@R1

001C1 MUL AB

001D1 ADD A,R3

001E1 MOV R3,A

001F1 INC R0

00201 INC R1

00211 DJNZ R2,L0

00231 WAIT: SJMP WAIT

END

Data Memory

addr

0x00

0x00

value

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	25	2D	00	05	00	00	00	00	00	00	00	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	01	01	01	01	01	00	00	00	00	01	01	01	01	01	00	00
30	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

3. bonus_2:(待demo, 結果尚未確認, 需要學校的微算機開發(實驗)版, 所以僅先放上作法)

(1) 作法:

- 程式也是從基本的lab01下去改。
- a. 程式寫法(與bonus01和lab01基本版不同的地方):
- b. 新增溢位檢查:再MUL AB之後, 用MOV A, B/JZ NOOV來判斷有沒有超過8-bit, 若B不等於0表示乘法結果大於255, 則執行INC R4, 將溢位次數部分加一。
- c. 增加一個register R4:在程式一開始先初始化為0(清空), 用來紀錄溢位的次數。
- d. LED顯示部分:R4=1 -> MOV P0, #0FEH -> LED亮一顆; R4>1 -> MOV P0, #0FCH -> 亮第二顆; 以此類推。去模擬”第一次超過亮一顆, 再超過一次多亮一顆的結果”。
- e. 程式最後:SJMP STAY讓程式停在顯示結果, 保持LED狀態。

(2) 程式&結果照片:

- 結果:最後在bonus02的部分:A[]從20H起, B[]從28H起, 兩個陣列(初始化都放32(20H));然後在MUL AB後的register A是存低位結果(0~FFH), register B是高位結果(判斷溢位);在P0(00H)部分的話, 它不是存A*B的結果, 而是會依溢位次數結果決定亮幾顆LED。

<pre> ORG 0000H 00001 START: MOV R0,#20H 00021 MOV R1,#28H 00041 MOV R2,#08H 00061 MOV A,#20H 000810: MOV @R0,A 00091 MOV @R1,A 000A1 INC R0 000B1 INC R1 000C1 DJNZ R2,10 000E1 MOV P1,#0FFH ;共陽極:全暗 00111 MOV R3,#00H 00131 MOV R4,#00H 00151 MOV R0,#20H 00171 MOV R1,#28H 00191 MOV R2,#08H 001B1 L0: MOV A,@R0 001C1 MOV B,@R1 001E1 MUL AB 001F1 MOV R7,A 00201 MOV A,B 00221 JZ NOOV 00241 INC R4 00251 NOOV: MOV A,R3 00261 ADD A,R7 00271 MOV R3,A 00281 INC R0 00291 INC R1 002A1 DJNZ R2,L0 002C1 MOV A,R4 002D1 JZ NOLED </pre>	<pre> 002F1 CINE A,#01H,CHK2 00321 MOV P1,#0FEH ;1次溢位=亮1顆 00331 SJMP STAY 00371 CHK2: CINE A,#02H,CHK3 003A1 MOV P1,#0FCH ;2次溢位=亮2顆 003D1 SJMP STAY 003F1 CHK3: CINE A,#03H,CHK4 00421 MOV P1,#0F8H ;3次溢位=亮3顆 00451 SJMP STAY 00471 CHK4: CINE A,#04H,CHK5 004A1 MOV P1,#0F0H ;4次溢位=亮4顆 004D1 SJMP STAY 004F1 CHK5: CINE A,#05H,CHK6 00521 MOV P1,#0E0H ;5次溢位=亮5顆 00551 SJMP STAY 00571 CHK6: CINE A,#06H,CHK7 005A1 MOV P1,#0C0H ;6次溢位=亮6顆 005D1 SJMP STAY 005F1 CHK7: CINE A,#07H,CHK8 00621 MOV P1,#08H ;7次溢位=亮7顆 00651 SJMP STAY 00671 CHK8: MOV P1,#00H ;8次溢位=亮8顆 006A1 SJMP STAY 006C1 NOLED: MOV P1,#0FFH ;沒溢位=全暗 006F1 STAY: SJMP STAY ;停在這裡顯示 END </pre>	
---	---	--

code在學校實作時要改成P0(學校是用P0)